

Onboarding DS — canal MQTT agent ↔ cloud (dev)

Guide pour le DS (Raspberry Pi). La référence complète du canal est `docs/MQTT.md` ; (Confère Gildas) ce document est le mode d'emploi : comment se connecter, ce qui est figé, ce qui reste ouvert.

1. Deux brokers, deux rôles



- **Mosquitto embarqué sur le Pi** — côté équipements du site (spec 18 §4.1.3). C'est votre terrain : `edge/deploy/mosquitto.conf` + `mosquitto.acl` (exemples, à valider sur boîtier).
⚠ l'exemple annonce `tls_version tlsv1.2` : la spec demande TLS 1.3 — à aligner.
- **EMQX côté cloud** — imposé par le CDC §3.3, éprouvé au lot 5 (ACL testées, TLS 1.3). Vous n'avez pas besoin de le maîtriser : c'est du MQTT 5 standard, `mosquitto_pub/sub` fonctionnent dessus pour vos tests.
- **Pas de bridge broker↔broker** : c'est l'agent (`h2s_agent`) qui porte le flux entre les deux.

2. Accès dev

	Valeur
Hôte	<code>mqtt.dev.h2splug.com</code>
Port	<code>8883</code> — TLS 1.3 seul (TLS 1.2 est refusé)
Certificat	Let's Encrypt (chaîne standard ISRG Root X1 — le trust store système suffit, <code>ca_file</code> facultatif)
Compte	<code>uuid:motdepasse</code> — transmis par le devops, un compte par boîtier enrôlé
Client ID	le même <code>uuid</code> que le compte

Le `uuid` du compte **est** l'identifiant de l'agent (`edge_clients.agent_uuid`) rendu à l'enrôlement. Mot de passe dans un fichier `0600` (`password_file`), jamais dans `agent.json`.

Ports 1883 (clair) et 18083 (console) = `127.0.0.1` seulement — invisibles depuis l'extérieur.

3. Plan de sujets — figé, à adopter

Le plan `energy/<device_id>/...` du cahier edge **n'est pas retenu** : l'espace se découpe par agent (l'ACL s'appuie dessus). Un agent n'existe que sous `h2s/agents/<son-uuid>/...`.

Sujet	Sens	QoS	Type d'enveloppe
<code>h2s/agents/<uuid>/telemetry</code>	agent → cloud	1	<code>live</code> (instantané site ~5 s) · <code>raw</code> (relevés bruts)
<code>h2s/agents/<uuid>/comptes-rendus</code>	agent → cloud	1	<code>records</code> · <code>command_result</code>
<code>h2s/agents/<uuid>/etat</code>	agent → cloud	1	<code>state</code> (version, buffer, horloge, sonde TPM)
<code>h2s/agents/<uuid>/ordres</code>	cloud → agent	1	<code>decision</code> · <code>command</code> — signés par le cloud

- `retain = false` partout, pas de *last will* (un testament ne peut porter ni nonce ni horodatage frais → rejeté par le cloud).
- Sessions persistantes (`clean_session = false`), identifiants stables.

4. Règles d'accès — un agent ne voit que lui-même

`mqtt/acl.conf` (deny par défaut), éprouvé par 11 cas sur EMQX réel :

Action	Verdict
Publier sur ses <code>telemetry</code> / <code>comptes-rendus</code> / <code>etat</code>	✓
S'abonner à son <code>ordres</code>	✓
Publier/s'abonner sur le sous-arbre d'un autre agent	✗ (SUBACK 128 / drop)
S'abonner à <code>h2s/agents/#</code> , <code>#</code> , <code>\$SYS/#</code>	✗
Publier sur son propre <code>ordres</code>	✗ — ce serait forger un ordre de pilotage

Aucune connexion anonyme : un compte non déclaré ne se connecte pas.

5. Enveloppe et signature — agent → cloud

```
{ "v":1, "agent": "<uuid>", "ts":1758000000, "nonce": "<32 hex>", "payload": "<chaîne JSON exacte>", "sig": "<base64url>" }
```

`sig` = signature **Ed25519** (ou ECDSA P-256 si le TPM le porte) de la chaîne canonique `H2S-AGENT-V1` (D249), méthode `MQTT`, sujet à la place du chemin :

```
H2S-AGENT-V1
MQTT
h2s/agents/<uuid>/telemetrie
<ts>
<nonce>
<SHA-256 hexadécimal de payload>
```

`payload` est une **chaîne JSON exacte** (pas un objet renvoyé) : elle voyage et se signe telle quelle — ni le courtier ni un parseur ne peuvent en toucher un octet.

Contrôles côté cloud : horodatage dans ± 5 min · agent connu et non révoqué · signature vérifiée · **nonce jamais vu** (`edge_nonces`, purgé à l'heure) · sujet appartenant à l'agent · enveloppe ≤ 256 Ko · $\leq 5\,000$ messages/heure/agent.

Horloge : la fenêtre ± 5 min impose un NTP fiable sur le Pi (chrony recommandé) — sinon rejets `perime` en série.

6. Ordres — cloud → agent

```
{ "v": 1, "type": "decision", "payload": "<chaîne JSON exacte>", "signature": "<base64url>", "key_id": "<clé des ordres>" }
```

Même charge signée que le sondage HTTP : signature de la clé des ordres du cloud, destiné au site de l'agent, validité cohérente, action connue, règles SEC1-6 appliquées à l'identique. Le transport ne décide rien.

Idempotence : un ordre remis deux fois (QoS 1 ou repli) n'est exécuté qu'une fois (`decision_cache` / `decision_id` ; `edge_messages.message_id` pour les enregistrements).

7. Mosquitto local (Pi) — votre périmètre

`edge/deploy/` livre : `mosquitto.conf` (8883 TLS sur le LAN équipements + 1883 loopback agent), `mosquitto.acl`, `agent.example.json`, `h2s-agent.service` (systemd durci). À faire :

1. Valider la config sur le Pi réel (marquée « non vérifiée sur boîtier ») ;
2. Passer `tls_version` à **1.3** ;
3. Équipements → votre broker local ; agent `h2s_agent` → cloud EMQX.

`agent.json` (extrait `cloud_mqtt`) :

```
"cloud_mqtt": {
  "host": "mqtt.dev.h2splug.com",
  "port": 8883,
  "tls": true,
  "ca_file": "/etc/h2s-agent/ca-courtier.pem",
  "password_file": "/etc/h2s-agent/mqtt.secret"
}
```

Sans `host`, le canal n'est pas ouvert (comportement d'avant, sondage HTTP seul).

8. Réponses au questionnaire devops (point par point)

#	Question	Réponse
1	FQDN DEV/TEST/PROD	<code>mqtt.dev.h2splug.com</code> (actif) ; <code>mqtt.test.</code> / <code>mqtt.</code> réservés en DNS, envs pas encore portés
2	8883 obligatoire	Oui — TLS 1.3 seul, vérifié
3	CA + credential client	LE standard + username/password (uuid:secret). Pas de mTLS aujourd'hui — si exigé, à décider côté infra
4	Client-ID	<code>uuid</code> d'enrôlement (= compte)
5	Topics + ACL	<code>h2s/agents/<uuid>/...</code> figé, ACL par agent (§3-4)
6	Consommateur aval	<code>agent-sync</code> (client <code>h2s-agent-sync</code>) — pas encore déployé en dev, voir §9
7	QoS / keepalive / retained	QoS 1 partout · session persistante · <code>retain=false</code> · pas de LWT
8	Taille max payload	Enveloppe $\leq$ 256 Ko côté cloud ; <code>max_packet_size</code> EMQX = défaut 1 Mio (à calibrer si gradients > 256 Ko — aujourd'hui la limite cloud tranche)
9	Versioning schéma	<code>v:1</code> + <code>type:</code> discriminé ; toute évolution = nouveau <code>v</code>
10	Cycle de vie des clés	Enrôlement par code (agent), comptes MQTT via rôle Ansible <code>mqtt</code> (prod) / <code>H2S_MQTT_AGENTS</code> (dev) ; révocation = retrait du compte + <code>edge_clients</code> révoqué
11	Registre de clés publiques	<code>edge_clients</code> (clé enrôlée Ed25519/ECDSA-TPM), vérif dans <code>AgentAuthenticator</code>
12	Fenêtre anti- rejeu	nonce unique + ± 5 min (<code>edge_nonces</code>)
13	Buffer hors- ligne	<code>h2s_agent/buffer.py</code> persistant sur le Pi — taille/expiry à préciser côté agent
14	Métriques	<code>h2s_mqtt_*</code> Prometheus (backend :9000) + <code>GET /api/agent/status</code> côté agent ; métriques EMQX à câbler (TODO)
15	Réseau	Sortant seul : TCP 8883 → <code>mqtt.dev.h2splug.com</code> (TLS natif, pas de proxy HTTP). Aucun flux entrant vers le Pi requis

9. État actuel en dev — à connaître

-  Broker joignable, TLS 1.3, auth + ACL éprouvés en live le 23/09/2026.
-  Un compte agent provisionné (uuid transmis séparément).
-  **agent-sync (le consommateur) n'est pas déployé en dev** : vos messages sont acceptés par le courtier mais rien ne les consomme encore — signature/enveloppe invérifiables de bout en bout tant que le service ne tourne pas. La vérif signature se fera sur le premier déploiement microservices ou en test local avec l'équipe backend.
-  `mqtt-init` n'importe les comptes que sur base vide : ajouter un agent = compléter `H2S_MQTT_AGENTS` (var CI `DEV_H2S_MQTT_AGENTS`) puis `docker compose up -d --force-recreate mqtt-init emqx` sur le VPS.

10. Checklist première connexion

1. `openssl s_client -connect mqtt.dev.h2splug.com:8883 -servername mqtt.dev.h2splug.com`
→ cert LE OK, TLS 1.3.
2. `mosquitto_sub -h mqtt.dev.h2splug.com -p 8883 --tls-version tlsv1.3 -u <uuid> -P <secret> -i <uuid> -t 'h2s/agents/<uuid>/ordres' -q 1` → SUBACK accordé.
3. `mosquitto_sub ... -t 'h2s/agents/#'` → refus (128) attendu.
4. `mosquitto_pub ... -t 'h2s/agents/<uuid>/telemetrie' -m '<enveloppe signée>'` → accepté.
5. `mosquitto_pub ... -t 'h2s/agents/<uuid>/ordres' -m 'x'` → refusé (drop, log `AUTHZ` côté broker).